BÀI TẬP THỰC HÀNH: XÂY DỰNG AI PHÂN LOẠI TRANG PHỤC VỚI FASHION-MNIST

I. ĐỀ BÀI (TÌNH HUỐNG THỰC TẾ)

Bối cảnh: Bạn vừa được nhận vào làm Kỹ sư AI cho một nền tảng thương mại điện tử lớn (như Shopee, Zalora). Mỗi ngày, nền tảng này nhận được hàng nghìn hình ảnh sản phẩm do người bán tải lên. Tuy nhiên, người bán thường quên chọn danh mục cho sản phẩm (áo, quần, giày, túi xách...).

Nhiệm vụ của bạn: Xây dựng một mô hình Mạng Nơ-ron Tích chập (CNN) để tự động nhận diện và phân loại hình ảnh sản phẩm vào đúng 10 danh mục thời trang.

Thông tin Bộ dữ liệu (Fashion-MNIST):

- Gồm 70.000 ảnh trắng đen (xám), kích thước 28×28 pixel.
- **10 nhãn (Labels) tương ứng từ 0 đến 9:** 0: Áo thun (T-shirt/top), 1: Quần dài (Trouser), 2: Áo len (Pullover), 3: Đầm (Dress), 4: Áo khoác (Coat), 5: Sandal, 6: Áo sơ mi (Shirt), 7: Giày sneaker, 8: Túi xách (Bag), 9: Ủng (Ankle boot).

Mục tiêu cần đạt: Mô hình của bạn phải đạt độ chính xác trên tập dữ liệu kiểm tra (Test Accuracy) tối thiểu **88%**.

II. HƯỚNG DẪN THỰC HÀNH TỪNG BƯỚC

Bước 1: Khởi tạo và tải dữ liệu

Fashion-MNIST đã được tích hợp sẵn trong Keras. Đầu tiên, hãy gọi nó ra và tạo một danh sách tên các lớp để tiện cho việc hiển thị sau này.

Python

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np

# Tải bộ dữ liệu
fashion_mnist = tf.keras.datasets.fashion_mnist
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()

# Danh sách tên các nhãn (tương ứng với số từ 0 đến 9)
class_names = ['Áo thun', 'Quần dài', 'Áo len', 'Đầm', 'Áo khoác',
               'Sandal', 'Áo sơ mi', 'Sneaker', 'Túi xách', 'Ủng']
```

Bước 2: Tiền xử lý dữ liệu (Preprocessing)

Giống như mọi khi, mạng nơ-ron cần dữ liệu đã được chuẩn hóa và định hình đúng khối 3D.

1. **Chuẩn hóa:** Chia các giá trị pixel trong x_train và x_test cho 255.0 để đưa về khoảng [0, 1].
2. **Định hình (Reshape):** Chuyển kích thước mảng từ (60000, 28, 28) thành khối (60000, 28, 28, 1) vì đây là ảnh xám 1 kênh màu.

Python

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

x_train = x_train.reshape((60000, 28, 28, 1))
x_test = x_test.reshape((10000, 28, 28, 1))
```

Bước 3: Thiết kế Kiến trúc CNN (Model Design)

Trang phục có nhiều chi tiết hơn chữ số, nên mạng CNN của chúng ta cần một lớp Dense lớn hơn một chút để xử lý thông tin, và thêm một lớp Dropout để chống học vẹt (Overfitting).

Gợi ý xây dựng kiến trúc models.Sequential():

- **Block 1:** Conv2D (32 filters, size 3x3, activation='relu') -> MaxPooling2D (size 2x2).
- **Block 2:** Conv2D (64 filters, size 3x3, activation='relu') -> MaxPooling2D (size 2x2).
- **Flatten:** Duỗi phẳng dữ liệu.
- **Khối phân loại (FC):** Thêm một lớp Dense với 128 nơ-ron (activation='relu').
- **Dropout:** Thêm lớp Dropout(0.3) để tắt ngẫu nhiên 30% nơ-ron khi huấn luyện, giúp mạng học các đặc trưng tổng quát hơn thay vì nhớ cục bộ.
- **Lớp Output:** Lớp Dense cuối cùng với 10 nơ-ron (activation='softmax').

Bước 4: Biên dịch và Huấn luyện (Compile & Train)

Cài đặt cho mô hình cách tự học và tự chấm điểm.

Python

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Bắt đầu huấn luyện (Thử nghiệm với 10 epochs)
history = model.fit(x_train, y_train, epochs=10,
                    validation_split=0.2, batch_size=64)
```

Bước 5: Đánh giá mô hình

Sau khi huấn luyện xong, hãy dùng tập x_test để xem mô hình đạt được bao nhiêu phần trăm độ chính xác.

Python

```
test_loss, test_acc = model.evaluate(x_test,  y_test, verbose=2)
print(f'\nĐộ chính xác trên tập kiểm tra: {test_acc*100:.2f}%')
```

(Nếu Test Accuracy ≥ 88%, xin chúc mừng, bạn đã hoàn thành mục tiêu đề bài!)

Bước 6: Trực quan hóa dự đoán (Trải nghiệm thực tế)

Hãy lấy một bức ảnh ngẫu nhiên trong tập kiểm tra và xem mô hình của bạn có nhận diện đúng nó là áo hay quần không. Chú ý sử dụng mảng class_names đã tạo ở Bước 1 để hiển thị tên thay vì hiển thị con số khô khan.

Python

```
import random
idx = random.randint(0, len(x_test) - 1)
sample_img = x_test[idx]
true_label_name = class_names[y_test[idx]] # Lấy tên thực tế

# Dự đoán
predictions = model.predict(np.expand_dims(sample_img, axis=0))
predicted_label_name = class_names[np.argmax(predictions)] # Lấy tên dự đoán

# Vẽ ảnh
plt.figure(figsize=(3,3))
plt.imshow(sample_img.reshape(28, 28), cmap='gray')
color = 'blue' if predicted_label_name == true_label_name else 'red'
plt.title(f"Thực tế: {true_label_name}\nDự đoán: {predicted_label_name}", color=color)
plt.axis('off')
plt.show()
```

III. THỬ THÁCH NÂNG CAO

Dành cho các bạn đã hoàn thành xong yêu cầu cơ bản:

1. **Vẽ biểu đồ Loss và Accuracy:** Dùng thư viện matplotlib để vẽ biểu đồ đường (line chart) thể hiện sự thay đổi của loss và val_loss qua từng Epoch dựa vào biến history ở Bước 4. Mô hình của bạn có đang bị Overfitting không?
2. **Đổi cấu trúc mạng:** Thử thêm một Block tích chập thứ 3 (với 128 filters) trước khi Flatten. Độ chính xác có tăng lên không, hay thời gian chạy lâu hơn mà kết quả không đổi?